



DPUSGEDistortionBezier5Ext

Distorts the image of a SGEView using two-dimensional Bezier interpolation.

1. Dynamic parameters:

<i>NDX, NDY</i>	Coordinates (in the 2D overlay coordinate system) where the top-left edge of the undistorted scenery is placed, if an additional undistorted output is desired. (See below)
<i>NDW, NDH</i>	Width and height (in pixels) of the additional undistorted output.
<i>TexSMin, TexSMax</i>	Specifies which range of the computed image (in horizontal dimension) is actually displayed. Default: (0...1), i.e. the whole computed image is displayed.
<i>TexTMin, TexTMax</i>	Specifies which range of the computed image (in vertical dimension) is actually displayed. Default: (0...1), i.e. the whole computed image is displayed.
<i>BorderLeft, BorderRight</i>	Number of grid points on the left and right side along which the image is faded to black. Useful for soft edge blending. Default: 0, which means a hard border is created.
<i>BorderTop, BorderBottom</i>	Number of grid points on the top and bottom side along which the image is faded to black. Useful for soft edge blending. Default: 0, which means a hard border is created.
<i>ShowControls</i>	Optionally displays the control points and their connecting lines on top of the scenery. (Default = 0: control points are invisible)
<i>ShowGrid</i>	Optionally displays the grid points and their connections on top of the scenery. (Default = 0: grid points are invisible)
<i>SaveObject</i>	Upon changes from 0 to 1, the SGE object which is internally used for the distortion is saved to the top of the installation directory.

2. Static parameters:

<i>ViewID</i>	The number of the view whose image is distorted. Counting starts at 0, which is assigned to the first view that is listed in the configuration.
<i>Port</i>	The TCP port number used for the network server. The default value normally doesn't need to be changed, unless two instances of DPUSGEDistortionBezier5Ext are running on the same computer.
<i>ResolutionX, ResolutionY</i>	Size of the off-screen image buffer (in pixels), into which the scenery is rendered. Should correspond to the image size after distortion. Large Resolution values reduce the achievable frame rate. Note: For older graphics boards (e.g. NVidia GeForce 6000, 7000 series), powers of two must be used for ResolutionX and ResolutionY, otherwise no output can be seen. Newer boards can use arbitrary values.



<i>SubdivX, SubdivY</i>	Subdivision of the projection geometry in X and Y direction. The more the image is distorted, the more subdivisions are required; however, that reduces the achievable frame rate. The default value is 50 and is sufficient for most practical purposes.
<i>DistortResetViewport</i>	Selects whether the coordinates Xnn, Ynn and NDX ... NDH are relative to the whole window into which the view is rendered (1) or only to the selected viewport (0). The default value is 0 and only needs to be changed if an additional undistorted output is desired. (see below)
<i>Points</i>	A list containing coordinates of control points (see below)
<i>Offset</i>	A list containing coordinates of all subdivision points („grid points“, see below)

3. Dependencies to other DPUs:

DPUSGEDistortionBezier5Ext must run together with **DPUSGEWorld** and **DPUSGEView**.

The following Index values must be assigned:

`DPUSGEWorld.Index < DPUSGEDistortionBezier5Ext.Index < DPUSGEView.Index`

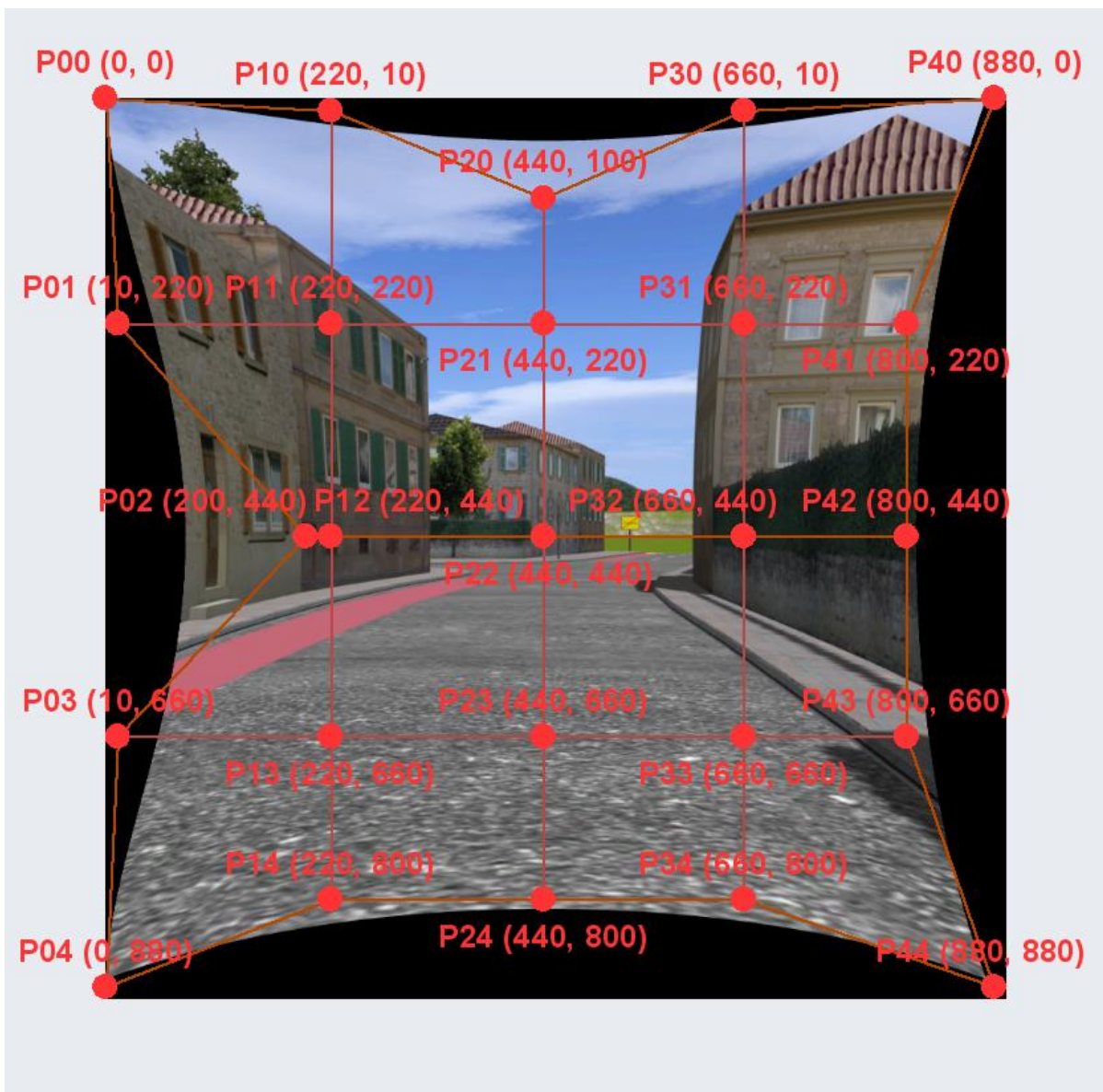
DPUSGEDistortionBezierParamsExt can be used to dynamically modify control and grid points over the network.

4. Functional principle:

DPUSGEDistortionBezier5Ext computes a two-dimensional Bezier patch based on 5x5 control points. The control points are read from the list **Points** (see configuration example below). Using the DPU input **ShowControls**, the control points can be shown during the simulation.

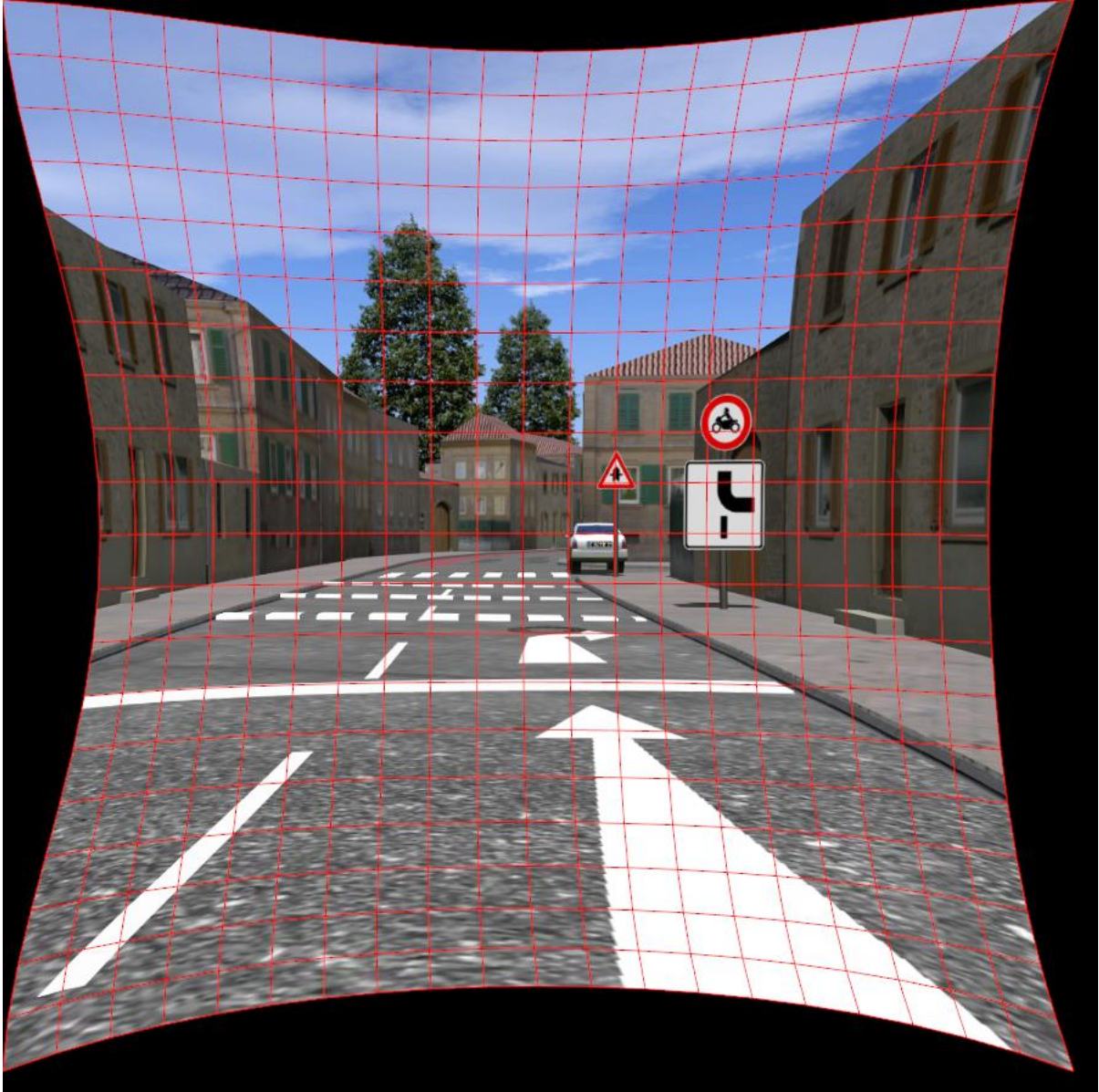
The result is a display area with curved borders. Please note that only the edge points (P00, P40, P04, and P44) are located exactly on the border of the displayed area. The other control points influence the Bezier curves more indirectly.

For an example set of control points, the following display area results:



In a second step, the Bezier patch surface is approximated by a grid of ($SubdivX * SubdivY$) partial surfaces (each a quadrilateral).

This fine grid is defined by $(SubdivX+1) * (SubdivY+1)$ „grid points“. It can be displayed using the DPU input *ShowGrid* and looks as following in this example:

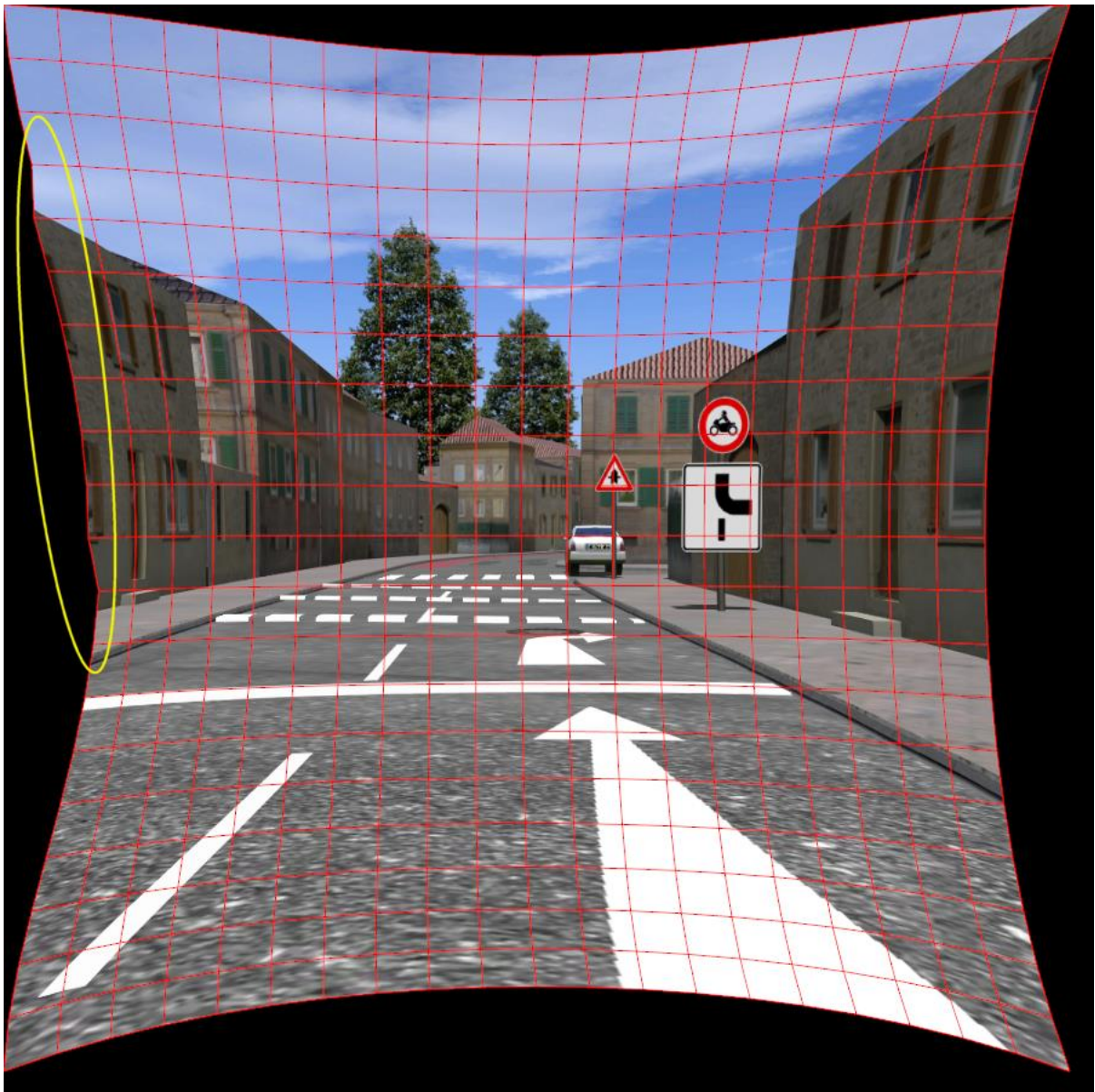


Even with the value $SubdivX = SubdivY = 20$ used here (which isn't particularly high), it is obvious that the quadrilaterals approximate the curved Bezier patch surface quite precisely.

In addition to configuring the control points, `DPUSGEDistortionBezier5Ext` allows to move individual grid points (i.e. the position of each intersection of two lines in the screenshot above) from their computed position. This method can be used to perform very fine corrections.

These "offset positions" are defined using the parameter `Offset` in the configuration.

In the following screenshot, the position of a few points of the first column (within the area marked by the yellow ellipsis) was moved to the left by 10 pixels.

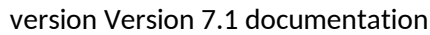


Assuming that the associated SGEView has a resolution of 900 x 900 pixels, the following example leads to the screenshot above:

```
DPUSGEDistortionBezier5Ext SGEDistortion
{
    Computer = {VIS};
    Index = 50;

    SubdivX = 20;
    SubdivY = 20;

    ResolutionX = 1024;
    ResolutionY = 1024;
```

[illegible]



```
        0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,
0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,
        0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,
0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,
        0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,
0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,
        0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,
0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,
        0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,
0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,
        0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,
0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,    0,0,
        );

};
```

5. Modifying distortion while the simulation is running:

The distortion parameters can be modified online (during the simulation) using `DPUSGEDistortionBezierParamsExt`, which provides a graphical, interactive interface for this purpose. The detailed procedure is described in its DPU documentation.

6. Additional undistorted output:

In many cases, it is useful to display not only the distorted scenery (on a projector), but additionally the undistorted scenery (on a computer monitor that has been connected in parallel).

`DPUSGEDistortionBezier5Ext` supports this configuration without a noticeable performance loss.

The necessary system and configuration settings for the undistorted output are described in the document for `DPUSGEDistortionQuad`. The same method also works for `DPUSGEDistortionBezier5Ext`.